SwarmPipe — Implementation Summary

Task 1 — Data Architecture Design

• Debezium CDC (PostgreSQL 14.23, pgoutput logical replication) captures changes from sites, buildings, assets, sensors, iot_telemetry, events into Kafka 4.3.1 topics (nectar_de.public.*)

• decimal.handling.mode=double set on the connector so Postgres NUMERIC columns arrive as plain doubles, not base64-encoded bytes

• PySpark 4.1.1 used as the single processing engine for both batch and streaming

• Batch: bounded spark.read, staged via CLI (source to bronze to silver to gold)

• Streaming: spark.readStream + foreachBatch, continuous micro-batch upserts, Spark-native checkpointLocation per entity

• Delta Lake 4.1.0 used as the streaming sink format to support row-level MERGE upserts

• Local filesystem data lake via DATA_LAKE_PATH (bronze/silver/gold/quarantine); moving to S3 would need real code changes (hadoop-aws package, credentials), not just an env var swap

• Data quality gate on every write: dedup + null-check, failing rows routed to a separate quarantine partition

• Streaming upserts are idempotent (Delta MERGE keyed on business keys), safe under checkpoint replay after a crash

Task 2 — Build a Data Pipeline

• Ingestion: batch (bounded Kafka read) and streaming (continuous readStream) both parse the Debezium 'after' JSON payload against a per-entity schema

• Validation: dropDuplicates for duplicate records, dropna for missing values; failing rows written to quarantine instead of Silver

• Transformation: four curated Gold aggregates, built once in a shared calculation helper and reused by both batch and streaming

• Hourly energy consumption — sum of power_consumption

• Average environmental conditions — avg of temperature, humidity, pressure, vibration

• Daily asset utilization — percentage of readings with operating_mode = running

• Fault statistics per asset — fault count plus High/Medium/Low severity breakdown

• Aggregation grain: site_id, building_id, asset_id (plus hour or day) for all four curated tables

• CLI supports batch (three staged commands: extract, transform-silver, transform-gold) and streaming (one continuous command per entity)

Task 3 — Data Modeling

• Dimension tables: dim_sites, dim_buildings, dim_assets, dim_sensors, each keyed by its own business key

• Fact tables: fact_iot_telemetry, fact_events (raw), plus four derived Gold facts (hourly_energy, hourly_environment, daily_utilization, daily_faults)

• ER diagram documented covering all six source tables and their relationships

• Partitioning: every layer partitioned by load_date=YYYYMMDD

• Batch partitions by the date the pipeline ran; streaming partitions fact records by the record's own timestamp date, so late or corrected data lands in the correct historical partition

• Delta transaction log provides file-level min/max column statistics for query pruning on streaming sinks

Task 4 — Multi-Asset Hierarchy & Connectivity

• Asset hierarchy modeled relationally via the self-referencing parent_asset_id column on assets

• 'Retrieve parent and child assets' implemented as a Flink SQL self-join on dim_assets

• LEFT JOIN used so root-level assets with no parent still appear in the result

• Site/customer context joined in via a dim_customer_site table

• Not implemented: assets-under-a-site, downstream-impacted-assets, orphan/disconnected-asset queries, and the NetworkX/Neo4j bonus

Task 5 — Data Quality Framework

• Implemented as part of the pipeline rather than a standalone framework

• Duplicate records removed via dropDuplicates on every batch and streaming write

• Missing values (nulls) detected via dropna and routed to a quarantine partition, separate from Silver

• Applied identically on both the batch path and the streaming micro-batch path

Task 6 — SQL Challenge

• All six requested queries written against the Gold/Silver tables built in Task 3

• Top 10 assets by energy consumption — from fact_hourly_energy

• Average daily energy consumption per site — rolled up from hourly to daily grain

• Assets with more than 10 faults in the last 30 days — from fact_daily_faults

• Assets with no telemetry in the last 24 hours — from raw fact_iot_telemetry, including assets with zero history

• Hourly utilization per building — computed fresh from raw telemetry, since the existing Gold table is asset+daily grain

• Sites with abnormal power consumption increases — mean + 2 standard deviation rule, since 'abnormal' is undefined in the spec